

Abstraction [Hiding the complexity] (Serious Polymorphism)

Showing the **essential details** without showing the **background details (implementation details)** is called **Abstraction**.

Example : Electric switch board, Car, Mobile phones,Lift, ATM machine and so on

* We can achieve abstraction in Java by using the following two concepts :

- Abstract class and abstract methods (0 - 100% abstraction i.e Partial Abstraction)
- interface (100 % abstraction i.e. Full Abstraction)

What is an abstract method ?

* An abstract method is a common method (General method) which does not contain any **method body so**, basically a method without any implementation (un-implemented method) is called abstract method.

* Every abstract method must have abstract keyword and it should not contain any method body, at the last of the method we must have terminator (;)

Example : public **abstract** void area();

Working with abstract class and abstract method :

```
class Vehicle
{
    public abstract void run();
}

class Car extends Vehicle
{
    void run()
    {
    }
}

class Bike extends Vehicle
{
    void run()
    {
    }
}

class Bus extends Vehicle
{
    void run()
    {
    }
}
```

Important Points :

- Whenever **action is common** but **implementations** are different then we should use abstract method.
- In Java, if any class contains at-least one abstract method then we should compulsory declare the class as an abstract class.
- We cannot create an object for abstract class.
- All the abstract methods of super class **must be overridden in all the sub classes otherwise sub class will become as an abstract class hence we cannot create an object for sub class also**

Abstract class and abstract methods :

A class that does not provide complete implementation (partial implementation) is defined as an abstract class.

An abstract method is a common/generic method which is used to provide easiness to the programmer because the programmer faces complexity to remember the method name.

An abstract method observation is very simple because every abstract method contains abstract keyword, abstract method does not contain any method body and at the end there must be a terminator i.e ; (semicolon)

In java, whenever action is common but implementations are different then we should use abstract method, Generally we declare abstract method in the super class and its implementation must be provided in the sub classes.

If a class contains at least one method as an abstract method then we should compulsory declare that class as an abstract class.

Once a class is declared as an abstract class we can't create an object for that abstract class.

*All the abstract methods declared in the super class must be overridden in the sub classes otherwise the sub class will become as an abstract class hence object can't be created for the sub class as well.

In an abstract class we can write all abstract methods OR all concrete methods or combination of both the method.

It is used to acheive partial abstraction that means by using abstract classes we can acheive partial abstraction(0-100%).

*An abstract class may or may not have abstract method but an abstract method must have abstract class.

Note :- We can't declare an abstract method as final, private and static (illegal combination of modifiers)

We can't declare an abstract class as a final.

//Programs :

```
abstract class Shape
{
    public abstract void draw();
}

class Rectangle extends Shape
{
    @Override
    public void draw()
    {
        IO.println("Drawing Rectangle");
    }
}

class Circle extends Shape
{
    @Override
    public void draw()
    {
        IO.println("Drawing Circle");
    }
}

public class ShapeDemo
{
    public static void main(String[] args)
    {
        Shape s = null;

        s = new Rectangle(); s.draw();
        s = new Circle(); s.draw();
    }
}
```

```
abstract class Car
{
    protected int speed = 100;

    public Car()
    {
        IO.println("Car class constructor");
    }

    public void getCarDetails()
    {
        IO.println("It has 4 wheels & 1 engine");
    }

    public abstract void run();
}

class Naxon extends Car
{
    public void run()
    {
        IO.println("Naxon car is running");
    }
}

public class IQ
{
    public static void main(String[] args)
    {
        Car car = new Naxon();
        IO.println("Car speed is : "+car.speed);
        car.getCarDetails();
        car.run();
    }
}
```